Prueba E06 — comando M: PC → ESP32 → servos STS3215 → ESP32 → PC

Un ciclo de control (request/response, 50 Hz en ros2_control): mover con SYNC_WRITE, leer con SYNC_READ y responder con la M de feedback

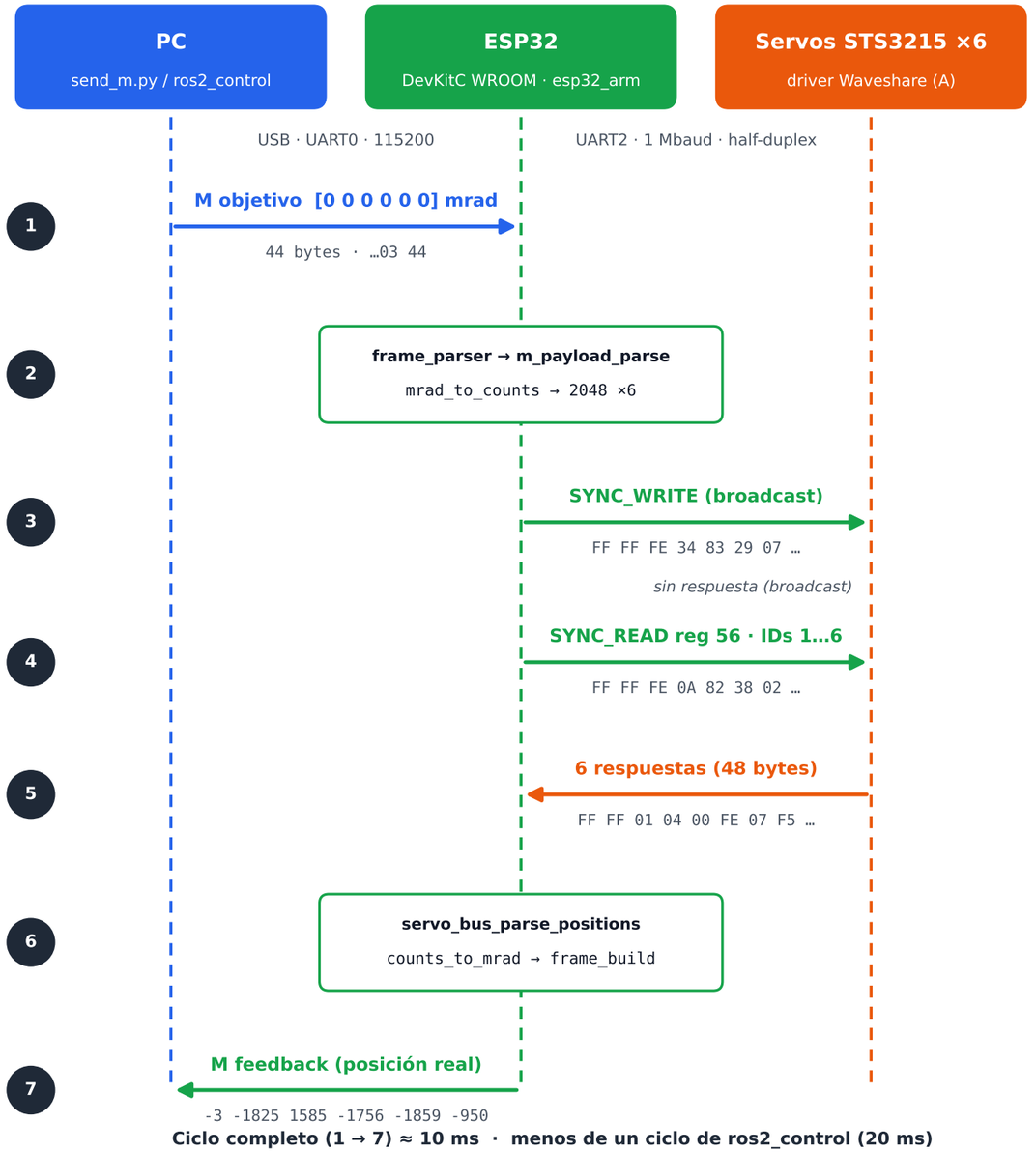
Trama M (PC ↔ ESP32)

Campo	Bytes	Descripción
STX	`02`	Inicio de trama
LEN	`30 30 33 39`	"0039": 4 dígitos ASCII
		= CID+FS+payload+ETX+LRC
CID	`4D`	'M' (motores)
FS	`1C`	Separador
payload	`2B 30 30 30 30`	j1 "+0000" (5 chars)
	`1C … 1C`	j2 … j6, separados por FS
ETX	`03`	Fin de trama
LRC	`44`	XOR de LEN … ETX

- UART0 (USB, CP2102) · 115200 8N1
- 44 bytes ≈ 3.8 ms por trama
- Joints en mrad, con signo, ±3142 (±π)
  orden: shoulder_pan … gripper → ID 1 … 6
- Misma trama en ambos sentidos:
  PC→ESP32 objetivo · ESP32→PC posición real
- Parser: busca STX, valida LEN (4…128),
  ETX y LRC; si falla descarta 1 byte

Ejemplo enviado (pose cero):

```
02 30 30 33 39 4D 1C 2B 30 30 30 30 1C
2B 30 30 30 30 … 1C 2B 30 30 30 30 03 44
```



Ciclo completo (1 → 7) ≈ 10 ms · menos de un ciclo de ros2_control (20 ms)

Paquete Feetech (ESP32 ↔ servos)

Campo	Bytes	Descripción
Header	`FF FF`	Inicio de paquete
ID	`FE / 01…06`	FE = broadcast, sin resp.
Length	`N + 2`	Instr. + parámetros + chk
Instr.	`83 / 82`	SYNC_WRITE / SYNC_READ
Params	…	Dependen de la instrucción
Checksum	`~(ID+…)`	~(ID+…+Params) & FF

Paquetes usados en la prueba

Paquete	Parámetros	Contenido
SYNC_WRITE	`29 07`	reg 41 (ACC), 7 bytes/servo
56 bytes	`ID acc pL pH`	objetivo (2048 = cero)
	`0 0 sL sH`	time 0 · vel. 300 pasos/s
SYNC_READ	`38 02 01…06`	reg 56 (pos. actual), 2 bytes
14 bytes		a los IDs 1…6, en ese orden
Respuesta	`FF FF ID 04`	una por servo, orden de ID
8 bytes c/u	`ERR lo hi chk`	ERR = 0 → sin errores

- UART2 (GPIO17 TX, GPIO16 RX) · 1 Mbaud 8N1
- Valores de 2 bytes en little-endian (STS)
- Ejemplo servo 1: FF FF 01 04 00 FE 07 F5
  → 0x07FE = 2046 cuentas → −3 mrad

Verificado con analizador lógico: CH0 = GPIO14 (consola, 115200) · CH1 = GPIO16 (respuestas del bus, 1 Mbaud)